

Informe de Exploración y Evaluación de Trayectorias Profesionales Alternativas asistido por LLM

“La locura es hacer la misma cosa una y otra vez
esperando obtener diferentes resultados.”

— Albert Einstein

Nombre: Jery Rodríguez Fernández

Grupo: C312

Telegram: [@JeryRf03](#)

Temática:

Desarrollar un sistema que, mediante técnicas y herramientas presentadas en la asignatura, resuelva un problema de optimización, planificación o satisfacción de restricciones, e integre un modelo de lenguaje (LLM) como parte funcional del sistema.

Exploración de trayectorias profesionales alternativas Dado un conjunto de decisiones posibles en una carrera profesional, generar y evaluar múltiples trayectorias válidas bajo distintos criterios. El sistema debe utilizar un LLM para analizar y comparar cualitativamente las trayectorias generadas.

NOTA: Es imprescindible que el lector consulte el archivo README.md adjunto al proyecto. Allí encontrará las instrucciones detalladas para su ejecución, así como la lista de dependencias necesarias, la estructura del proyecto y una vista rápida de la interfaz de usuario (UI) ilustrada con un caso de prueba.

Colección de Datos:

En respuesta a las siguientes preguntas:

- ¿Cómo está estructurado el dataset? ¿Es generado u obtenido?

- ¿El dataset utilizado —ya sea obtenido o generado— realmente cumple con las condiciones necesarias para el problema definido?

... se expone la siguiente descripción:

En primer lugar, cabe destacar que la recopilación de datos se centralizó en un entorno profesional vinculado a la informática, el software, las tecnologías, el marketing, el diseño y demás áreas afines.

El dataset final denominado “normalized_courses.json” se construye a partir de la integración y normalización de cuatro fuentes originales provenientes de Kaggle, las cuales contienen información sobre cursos en línea de distintas plataformas (Skillshare, Coursera, Udemy y una colección multiplataforma). Las fuentes se presentan a continuación:

- <https://www.kaggle.com/datasets/everydaycodings/multi-platform-online-courses-dataset>
- <https://www.kaggle.com/datasets/mahmoudahmed6/skillshare-top-1000-course>
- <https://www.kaggle.com/datasets/khusheekapoor/coursera-courses-dataset-2021>
- <https://www.kaggle.com/datasets/yusufdelikkaya/udemy-online-education-courses>

Mediante un proceso automatizado implementado en el script `download_dataset.py`, los archivos CSV crudos almacenados en `data/csv/` se consolidan en un esquema canónico único con la siguiente estructura estandariza:

- **name:** el título del curso tal como aparece en la plataforma.
- **duration_months:** tiempo estimado para completarlo.
- **difficulty:** escala numérica de dificultad en un rango de 1 a 5.
- **category:** categoría temática.
- **cost_usd:** precio en dólares (0 significa que es gratuito).
- **level:** etiqueta textual del nivel ("Beginner", "Intermediate", "Advanced").
- **skills:** lista de las habilidades que el usuario adquiere al completar el curso.

- **link:** la URL para acceder al curso directamente.

El resultado es un dataset homogéneo, listo para ser utilizado en tareas como construcción de rutas de aprendizaje.

De igual forma, se genera un `embedding.json`. Este archivo almacena representaciones vectoriales (embeddings) del nombre de cada curso, producidas con spaCy.

```
{ { "name": "curso", "embedding": [0.12, -0.34, ...]}, ... }
```

El objetivo de generar esta estructura es capturar la esencia semántica de cada curso en forma de vector numérico. Esto permite que, más adelante, el sistema pueda comparar cursos no solo por palabras clave exactas, sino por su significado real — de modo que una búsqueda de trayectoria encuentre cursos verdaderamente relevantes, aunque usen vocabulario distinto.

El dataset construido cumple plenamente con las condiciones para dar solución a nuestra búsqueda de trayectorias profesionales porque, en primer lugar, cada curso registrado incluye de manera explícita y estructurada todos los atributos necesarios para evaluar y planificar rutas de aprendizaje: costo en USD, duración en meses, nivel de dificultad en una escala numérica estandarizada y una lista detallada de habilidades enseñadas. Esta completitud de atributos permite que cualquier algoritmo o sistema de recomendación pueda tomar decisiones fundamentadas sobre qué cursos agrupar, en qué orden sugerirlos y con qué restricciones presupuestarias o temporales.

En segundo lugar, el hecho de que los datos se hayan normalizado bajo un esquema canónico común —es decir, con encabezados, formatos y escalas homogéneas— facilita enormemente tareas como la indexación, la búsqueda por rangos y la recuperación eficiente de información, evitando la complejidad de lidiar con estructuras heterogéneas propias de las fuentes originales.

Por último, el uso de “embeddings” generados a partir de los títulos de los cursos introduce una capa de inteligencia semántica que va más allá de la coincidencia textual exacta: permite comparar el objetivo de aprendizaje de un usuario (expresado en lenguaje natural, por ejemplo "aprender fundamentos de ciencia de datos") con los títulos de los cursos disponibles, encontrando aquellos que sean conceptualmente

afines, aunque no compartan palabras idénticas. Esta capacidad mejora sustancialmente la calidad de las recomendaciones, haciendo que el sistema sea más flexible, robusto y alineado con las intenciones reales del usuario.

Módulos Implementados:

A continuación, se detalla el papel que desempeña cada módulo en el sistema y de esta forma respondiendo a preguntas como:

- ¿Cómo se modela el problema? ¿Cómo está diseñado el algoritmo? ¿Cuáles son sus principales componentes y flujo de ejecución?
- ¿Qué rol cumple el LLM dentro del sistema? ¿En qué etapas o tareas específicas se utiliza?
- ¿Cómo se modela formalmente el problema? ¿Qué representaciones matemáticas o computacionales se utilizan (grafos, árboles, embeddings, etc.)?

`app.py`

Es la interfaz de usuario principal basada en Streamlit. Carga los cursos normalizados, permite al usuario ingresar un objetivo, seleccionar habilidades previas y elegir un criterio de optimización, y luego llama al planner y al adaptador LLM para generar rutas de aprendizaje. También renderiza los resultados con métricas, enlaces a los cursos y explicaciones de comparación generadas por el LLM. Dicho de otro modo, este componente cumple la función de orquestador del sistema.

`download_dataset.py`

Constituye el script de ingestión de datos. Su objetivo es descargar o copiar conjuntos de datos desde Kaggle, normalizar filas de CSV a un esquema común (`normalized_courses.json`) y luego generar un caché de “embeddings” con `llm_adapter.generate_embeddings`. Contiene lógica para reconocer cabeceras aliadas, limpiar campos de duración, costo, dificultad, nivel y habilidades, y producir

datos listos para el planner. En términos más simples: este módulo se encarga de bajar los datos, limpiarlos y dejarlos listos para que el planificador los use.

`llm_adapter.py`

Aquí es donde el LLM desempeña su rol fundamental: interpreta la intención del usuario, justifica la relevancia de cada ruta, evalúa y compara alternativas para recomendar una solución óptima, e infiere los prerrequisitos necesarios para alcanzar un objetivo de aprendizaje:

- **`pick_model`**: detecta el idioma del texto usando “`langdetect`” y devuelve el modelo `spaCy` apropiado (“`es_core_news_md`” para español, “`en_core_web_md`” para inglés). Se usa para elegir el modelo correcto al generar “`embeddings`” de texto.
- **`ask_llm`**: envía un “prompt” al cliente de Gemini (`google.genai.Client`) usando el modelo configurado y devuelve la respuesta como texto limpio. Es la capa de llamada general al LLM.
- **`parse_input`**: El sistema recibe texto libre ingresado por el usuario y, mediante un “prompt” cuidadosamente diseñado, le indica al LLM que extraiga y estructure la información en un objeto JSON que contenga los siguientes campos: `goal`, `preferences`, `skills` y `avoids`.
- **`explain_paths_brief`**: para cada ruta generada, se construye un “prompt” que incorpora el objetivo, los cursos, el tiempo estimado, el costo y el nivel de dificultad. Luego, se solicita al LLM que genere una explicación breve que justifique cómo esa ruta contribuye a alcanzar el objetivo planteado. Como resultado, se devuelve una lista con las explicaciones concisas correspondientes a cada ruta.
- **`explain_comparison`**: a partir del perfil del usuario y de las rutas generadas, se construye un “prompt” que solicita al LLM una comparación cualitativa entre dichas rutas y una recomendación fundamentada. El sistema devuelve una única explicación en lenguaje natural que justifica cuál es la opción más adecuada para el usuario.

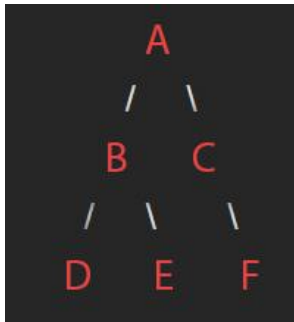
- **get_text_embedding:** convierte un texto en un vector denso usando el modelo spaCy elegido por idioma. Devuelve el “embedding” como lista de flotantes, o None si falla.
- **generate_embeddings:** crea y guarda “embeddings” para todos los cursos en `normalized_courses.json`. Si ya existe `embedding.json`, no vuelve a generarlas. Recorre cada curso, llama a `get_text_embedding` y escribe el resultado en disco.
- **infer_prerequisites_for_objective:** consulta al LLM para identificar los prerequisites necesarios para un objetivo de aprendizaje determinado. Como respuesta, se obtiene una lista breve de entre 1 y 3 habilidades o temas requeridos. Si el objetivo es suficientemente básico como para no requerir conocimientos previos, se retorna una lista vacía.

`planner.py`

Representa el núcleo lógico del sistema de recomendación de rutas de aprendizaje. A partir de un objetivo y las habilidades previas del usuario, este módulo se encarga de indexar los cursos disponibles, inferir prerequisites y construir árboles de dependencias que luego aplanan en rutas ordenadas y optimizadas. Para ello combina búsqueda léxica, similitud semántica mediante “embeddings” y llamadas al LLM, permitiendo generar múltiples rutas alternativas evaluadas según criterios como coste, duración o dificultad.

CourseNode: estructura de árbol que modela cursos con relaciones de dependencia. Cada nodo representa un curso, y sus nodos hijos corresponden a los prerequisites que deben completarse antes de poder tomar el curso asociado al nodo padre.

- **flatten:** convierte el árbol de prerequisites en una lista lineal de nombres de cursos en orden “post-order”.
- **print_tree:** imprime el árbol de curso con detalles legibles sobre habilidades satisfechas, alternativas y “skills” no resueltas.



Sea A el nodo que representa el curso objetivo, cuyos conocimientos se desean adquirir. El grafo acíclico dirigido (DAG) que modela la relación de prerequisites necesarios para alcanzar A puede representarse mediante una secuencia topológicamente ordenada de nodos: $D \rightarrow E \rightarrow B \rightarrow F \rightarrow C \rightarrow A$.

Funciones

- **load_courses:** carga el archivo `normalized_courses.json` y devuelve la lista de cursos normalizados. Si falla, retorna lista vacía.
- **_load_embeddings_data:** lee `embedding.json` y devuelve un diccionario de “embeddings” por nombre de curso, evitando datos corruptos o inexistentes.
- **index_courses:** construye un índice de cursos por nombre. Si hay duplicados, mantiene la entrada que tenga más habilidades, para preferir registros más ricos.
- **build_skill_index:** construye un índice inverso que mapea cada habilidad “skill” al conjunto de nombres de los cursos en los que dicha habilidad es abordada.
- **_score_course_for_criterion:** define una heurística que calcula una puntuación de optimización para cada curso de forma individual, según el criterio seleccionado por el usuario: priorizar bajo costo (cheapest path), priorizar corta duración (fastest path) o un equilibrio balanceado entre duración, dificultad y costo. La heurística asigna pesos fijos y diferenciados a costo, duración estimada y nivel de dificultad de acuerdo con el criterio elegido. La puntuación obtenida — donde valores más bajos indican mayor optimalidad— permite ordenar los cursos de mejor a peor según el criterio elegido, priorizando así unas dimensiones sobre otras.
- **_cosine_similarity:** calcula la similitud coseno entre dos “embeddings”.
- **_is_skill_covered:** determina si una habilidad “skill” ya está cubierta por el conjunto de habilidades conocidas del usuario. Para ello, se combinan dos estrategias: coincidencia léxica simple (por ejemplo, igualdad exacta o parcial de términos) y comparación semántica mediante “embeddings”, que permite

identificar habilidades equivalentes o muy relacionadas, aunque estén expresadas con distinto vocabulario.

- **`_passes_category_filter`**: rechaza aquellos cursos cuyo contenido presenta una similitud excesiva con las categorías que el usuario ha indicado explícitamente que desea evitar.
- **`find_course_by_skill`**: realiza una búsqueda híbrida de cursos para una habilidad determinada “skill”, combinando tres estrategias complementarias: indexado directo por nombre de la “skill”; similitud semántica basada en “embeddings”; y señales léxicas como coincidencia de términos clave en los títulos. La puntuación final se construye sumando estas señales con pesos predefinidos, aplicando también ligeros penalizadores. El resultado es una lista de cursos candidatos, relevantes y ordenados de mayor a menor puntuación agregada.
- **`_get_candidate_courses_for_skill`**: toma los candidatos devueltos por `find_course_by_skill`, aplica filtro de categoría y ordena según el criterio, devolviendo los mejores cursos.
- **`_infer_prereq_skills_for_topic`**: extiende inferencia de prerrequisitos vía LLM y guarda resultados en caché para evitar llamadas repetidas.
- **`_infer_prereq_skills_for_course`**: infiere prerrequisitos para un curso específico; los cursos de nivel “beginner” se consideran sin dependencias.
- **`_resolve_route_for_target_course`**: construye recursivamente el árbol de prerrequisitos para un curso objetivo. Para cada “skill” requerida no cubierta, busca cursos candidatos, guarda alternativas y crea subnodos. También detecta ciclos y marca “skills” no resueltas.
- **`generate_paths`**: es la función principal que orquesta la generación de rutas de aprendizaje. Primero indexa los cursos y construye un índice inverso de “skills” a nombres de curso, además de cargar “embeddings” precomputados. Luego identifica los cursos más relevantes para el objetivo de aprendizaje del usuario, filtrando aquellos que pertenecen a categorías que el usuario desea evitar y ordenándolos según el criterio de optimización elegido (costo, duración o balanceado). Para cada curso objetivo, resuelve recursivamente su árbol de

prerrequisitos hasta alcanzar las habilidades iniciales del usuario o hasta llegar a un curso de nivel principiante (el cual no requiere conocimientos previos), aplanando cada árbol a una lista ordenada de cursos (desde los más básicos hasta el curso objetivo), y calcula métricas agregadas como duración total en meses, costo total en USD y dificultad promedio. Finalmente, devuelve una lista de hasta “max_paths” rutas, cada una como un diccionario que contiene el curso objetivo, la secuencia ordenada de cursos y las métricas calculadas.

Metodología Experimental:

Más allá de los “tests” implementados (que ya validan el funcionamiento, la estructura y la composición del sistema), se ha creado un conjunto de datos independiente y específico con el fin de evaluar exclusivamente la calidad de las respuestas (y no el comportamiento estructural).

NOTA: Se recomienda consultar las imágenes disponibles en la carpeta “[previews](#)”, ubicada en la raíz del proyecto. Alternativamente, el lector también puede encontrar dichas imágenes en la última sección del archivo [README.md](#). Dichas imágenes representan un caso de prueba sencillo a modo de ejemplo base, en el que se puede observar cómo el sistema responde generando varias rutas posibles para alcanzar el objetivo deseado. También se aprecia la influencia del LLM, que explica su criterio para cada ruta y finalmente emite una recomendación sobre la trayectoria más acorde a los objetivos del usuario.

En la carpeta “[experiments](#)” se encuentra un conjunto de condiciones específicas para evaluar la calidad de las respuestas del sistema. Allí el lector encontrará un conjunto de cursos ([experiments.json](#)) diseñado para someter al motor de recomendaciones a pruebas de esfuerzo en los tres criterios de optimización admitidos: la ruta más rápida (minimiza el tiempo hasta la finalización), la ruta más económica (minimiza el costo total) y la ruta equilibrada (optimiza tanto el tiempo como el costo).

El conjunto de datos de prueba fue construido de manera intencional (no muestreado a partir de datos reales), con un tamaño de cursos restringidos para facilitar, manualmente, el reconocimiento de rutas posibles y así verificar que cada estrategia de

enrutamiento produzca recomendaciones semánticamente correctas y distinguibles entre sí.

Para ejecutar la prueba de calidad, se debe correr el siguiente comando desde la raíz del proyecto: `python -m experiments.quality_test`.

Análisis de los Resultados:

El sistema demostró ser capaz de generar trayectorias de aprendizaje con una estructura lógica y pedagógicamente sólida. Partiendo de un conjunto de habilidades iniciales y un objetivo final, el motor de planificación construyó rutas que respetan las dependencias de prerequisites gracias al uso del LLM para inferir conocimientos previos necesarios. En todos los casos evaluados, las rutas propuestas mostraban una progresión natural desde habilidades más básicas hacia conceptos más avanzados, sin saltos injustificados ni ciclos. Esto confirma que el enfoque recursivo de resolución de prerequisites, combinado con la inferencia semántica del LLM y el uso de grafos dirigidos acíclicos, produce trayectorias que un humano consideraría razonables y pedagógicamente válidas.

Cada uno de los tres criterios de optimización —ruta más rápida, más económica y equilibrada— generó recomendaciones claramente distinguibles entre sí. La ruta más rápida priorizó cursos de menor duración, aunque en algunos casos con costos más elevados. La ruta más económica minimizó el costo total, seleccionando cursos gratuitos o de bajo precio, a costa de alargar la duración total. La ruta equilibrada ofreció un compromiso sensato entre tiempo y dinero, sin extremar ninguna de las dos variables. Esta diferenciación valida que el sistema de puntuación híbrido (que combina relevancia semántica con criterios numéricos) es fiable y se comporta de manera satisfactoria.

El conjunto de datos de prueba fue construido intencionalmente con un tamaño reducido para facilitar la inspección humana de las rutas. Esto permitió detectar situaciones límite donde, alguna trayectoria generada se alejaba parcialmente del objetivo esperado. Ante estos escenarios, el LLM interviene en la fase de análisis

cualitativo: compara las rutas generadas, identifica sus fortalezas y debilidades, y recomienda al usuario la mejor opción posible entre todas las expuestas, incluso cuando ninguna es perfecta. Esta capacidad de razonamiento en contexto añade una capa de robustez a nuestro sistema.

Limitaciones y Posibles Mejoras:

Actualmente, los pesos y umbrales utilizados en el sistema de puntuación híbrido (por ejemplo, el balance entre relevancia semántica y criterios de optimización, o los coeficientes de normalización de costo, duración y dificultad) están fijados de manera analítica. Detrás de cada valor existe una lógica razonable y fundamentada, pero su determinación es estática y no se ajusta automáticamente al contexto ni a las preferencias particulares de cada usuario.

Una mejora significativa consistiría en implementar un algoritmo genético combinado con un sistema de retroalimentación del usuario. En lugar de fijar los parámetros a priori, se definiría una población inicial de configuraciones (cada una representando un vector de pesos). Al generar rutas para un usuario concreto, el sistema le presentaría varias opciones y este evaluaría cuál se ajusta mejor a sus expectativas. Esa valoración serviría como función de fitness: las configuraciones que producen rutas mejor valoradas tendrían más probabilidad de recombinarse y mutar en la siguiente generación.

De este modo, el sistema aprendería de forma iterativa el balance óptimo entre tiempo, costo, dificultad y relevancia semántica para cada perfil de usuario, en lugar de depender de valores fijados analíticamente con anterioridad. Esta estrategia permitiría una personalización mucho más fina y adaptativa, mejorando la calidad percibida de las recomendaciones sin necesidad de reingenierías manuales constantes.

El proyecto se diseñó bajo la temática de tecnología y software, que de por sí es bastante amplia y abarca muchos campos. Sin embargo, hubiera sido una excelente idea lograr abarcar también otras áreas profesionales. El problema radica en que el enfoque recursivo de resolución de prerequisites puede generar árboles de gran tamaño si el

objetivo requiere muchas habilidades intermedias, lo que dificultaría extender el sistema a dominios muy distintos sin una optimización adicional.

Por otro lado, el sistema requiere acceso a la API de Google Gemini y a los modelos de spaCy descargados localmente. Sin conexión a internet o sin las claves de API configuradas correctamente, el LLM no puede operar, lo que inutiliza la inferencia de prerrequisitos, la generación de explicaciones y la comparación cualitativa de rutas. Esta dependencia limita el despliegue del sistema en ciertos entornos.

"""

Se agradece al lector por emprender esta lectura.

Que el viento sople siempre a favor de quienes se atreven a leer,
y que la curiosidad, de una vez por todas, emprenda su vuelo.
Porque donde las palabras terminan,
comienza la libertad de quien las interpreta.

"""